

JAVASCRIPT ASYNCHRONOUS

- ❖ Synchronous (Đồng bộ) vs Asynchronous (Bất đồng bộ)
- ❖ Xử lý các thành phần bất đồng bộ nối tiếp
 - Callback
 - Promise
 - Async/await

ĐỒNG BỘ

- ❖ Javascript bản thân nó là một ngôn ngữ đồng bộ (synchronous) cũng thể gọi là đơn luồng, có nghĩa là tại một thời điểm nó chỉ có thể làm một công việc duy nhất, dẫn đến việc các dòng code được thực thi theo đúng trình tự mà nó được tạo ra;

```
console.log('line one');  
console.log('line two');  
console.log('line three');
```

statement 1

statement 2

statement 3

statement 4

BẤT ĐỒNG BỘ

- ❖ Asynchronous (bất đồng bộ/bất tuần tự): tức là code chương trình không hẳn tuần tự nữa, nhiều lệnh có thể thực hiện cùng lúc. Có khi lệnh dưới cho kết thúc và cho kết quả trước cả lệnh phía trên.

Trong javascript, các hàm `setTimeout`, `setInterval`, `fetch...` là tiêu biểu cho các hàm xử lý bất đồng bộ trong javascript

ưu điểm và nhược điểm

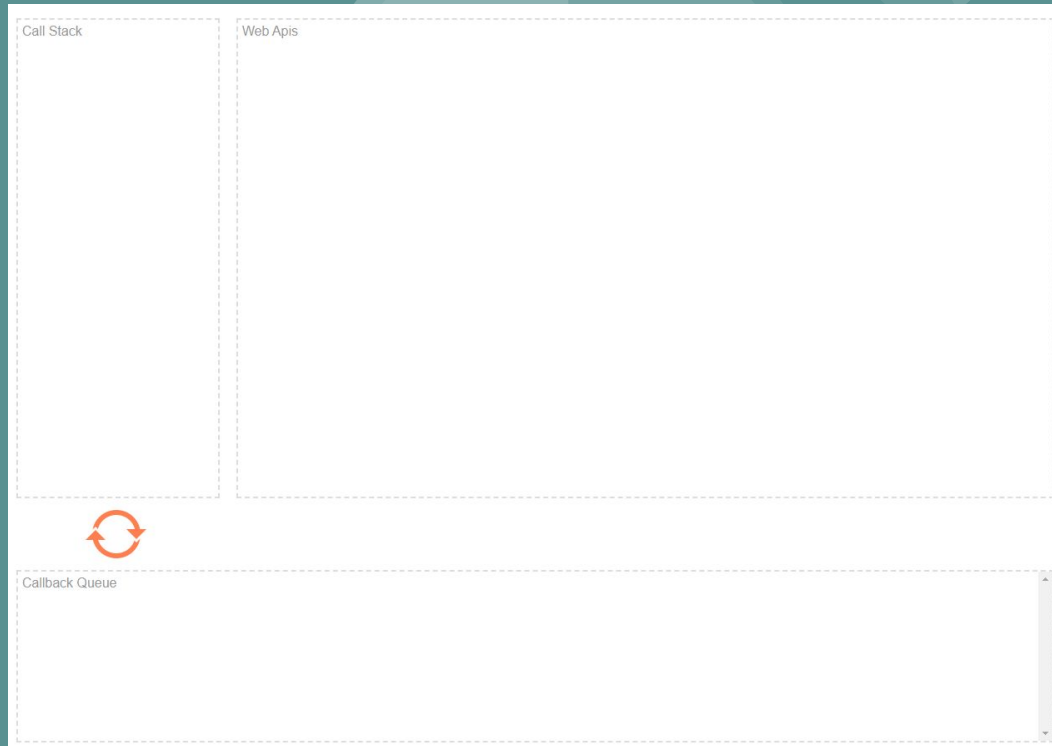
-Cái hay của đồng bộ là dễ kiểm soát quá trình xử lý. Cái hay thứ hai là dễ kiểm soát lỗi phát sinh. Còn điểm không hay của xử lý đồng bộ là gì? Đó là do chạy theo thứ tự nên sẽ có trạng thái chờ, lệnh trên chạy quá lâu sẽ làm ảnh hưởng đến các lệnh dưới.

- Cái hay của xử lý bất đồng bộ là gì? Đó là nó có thể tối ưu được sức mạnh của hệ thống, thứ hai giúp giảm thời gian chờ, giúp code chạy nhanh hơn. Còn điểm không tốt của bất đồng bộ? Không phải hệ thống nào cũng dùng bất đồng bộ được, thứ hai là khó làm quen xử lý và kiểm soát lỗi phát sinh.

BẤT ĐỒNG BỘ

cơ chế bất đồng bộ:

<http://latentflip.com/>



Xử lý Bất Đồng Bộ

Trong javascript có nhiều phương án xử lý bất đồng bộ :

Dùng hàm Callback

Sử dụng Promise

Dùng async/await (của es6)

Xử lý bất đồng bộ bằng callback

Hàm callback là một hàm được truyền vào như tham số cho 1 hàm khác.

```
congViec1(() => {  
  congViec2(() => {  
    congViec3(() => {  
      congViet4(() => {  
        congViec5(() => {  
          console.log("Hoàn Thành");  
        })  
      })  
    })  
  })  
});
```

Xử lý bất đồng bộ bằng callback

- ❖ Cú pháp callback tuy được việc tuy nhiên có tồn tại một số vấn đề khi sử dụng trong thực tế
 - Cú pháp hình bậc thang nhìn rất khó chịu và khó xác định xem code đang làm cái quái gì tại một thời điểm, gây khó khăn trong việc kiểm tra việc thực thi của code
 - Trong thực tế không phải mọi công việc dù bất đồng bộ hay đồng bộ đều thành công và ta cần phải xử lý lỗi xảy ra tại bất kỳ một mắt xích nào trong các công việc được làm lần lượt, gây khó khăn trong việc xử lý lỗi
 - Các công việc không phải lúc nào cũng được thực hiện một cách tuần tự mà có thể theo điều kiện, Ví dụ cả công việc B và C đều phụ thuộc vào kết quả của A, tuy nhiên tùy vào kết quả của A mà ta chỉ cần thực hiện B hoặc C hoặc cả 2, điều này vẫn có thể thực hiện với callback nhưng sẽ khá phức tạp và tương đối rối mắt

```
a(function (resultsFromA) {  
  b(resultsFromA, function (resultsFromB) {  
    c(resultsFromB, function (resultsFromC) {  
      d(resultsFromC, function (resultsFromD) {  
        e(resultsFromD, function (resultsFromE) {  
          f(resultsFromE, function (resultsFromF) {  
            console.log(resultsFromF);  
          })  
        })  
      })  
    })  
  })  
});
```

```
function register()  
{  
  if (!empty($_POST)) {  
    $msg = '';  
    if ($_POST['user_name']) {  
      if ($_POST['user_password_new']) {  
        if ($_POST['user_password_new'] === $_POST['user_password_repeat']) {  
          if (strlen($_POST['user_password_new']) > 5) {  
            if (strlen($_POST['user_name']) < 65 && strlen($_POST['user_name']) > 1) {  
              if (preg_match('/^[a-z\d]{2,64}$/i', $_POST['user_name'])) {  
                $user = read_user($_POST['user_name']);  
                if (!isset($user['user_name'])) {  
                  if ($_POST['user_email']) {  
                    if (strlen($_POST['user_email']) < 65) {  
                      if (filter_var($_POST['user_email'], FILTER_VALIDATE_EMAIL)) {  
                        create_user();  
                        $_SESSION['msg'] = 'You are now registered so please login';  
                        header('Location: ' . $_SERVER['PHP_SELF']);  
                        exit();  
                      } else $msg = 'You must provide a valid email address';  
                    } else $msg = 'Email must be less than 64 characters';  
                  } else $msg = 'Email cannot be empty';  
                } else $msg = 'Username already exists';  
              } else $msg = 'Username must be only a-z, A-Z, 0-9';  
            } else $msg = 'Username must be between 2 and 64 characters';  
          } else $msg = 'Password must be at least 6 characters';  
        } else $msg = 'Passwords do not match';  
      } else $msg = 'Empty Password';  
    } else $msg = 'Empty Username';  
    $_SESSION['msg'] = $msg;  
  }  
  return register_form();  
}
```



Xử lý bất đồng bộ bằng promise

Promise: là một cơ chế trong JavaScript giúp bạn thực thi các tác vụ bất đồng bộ mà không rơi vào callback hell là tình trạng các hàm callback lồng vào nhau ở quá nhiều tầng.

- hiểu đơn giản: promise nó là 1 lời hứa, nó có 2 tác vụ thành công và thất bại , nếu thành công nó sẽ làm 1 công việc nào đó và trả về `.then` , nếu thất bại nó sẽ làm 1 công việc nào đó và trả về `.catch`
- nó bao gồm 3 trạng thái
 - + Pending: tác vụ vẫn đang thực hiện
 - + Fullfilled: tác vụ thực hiện thành công
 - + Rejected: tác vụ thực hiện thất bại

Xử lý bất đồng bộ bằng promise

2: Đây là cách để tạo ra một Promise:

```
/// Cách tạo 1 promise
```

```
const promiseExample = new Promise((resolve, reject) => {  
  // resolve("hello Word");  
  reject("Thất bại");  
})
```

```
console.log(promiseExample);
```

```
promiseExample
```

```
.then((res) => {  
  console.log(res);  
})  
.catch((error) => {  
  console.log(error);  
})
```

```
// Function resolve sẽ gọi function trong .then và .finally
```

```
// Function gọiCatchVaFinally sẽ gọi function trong .catch và .finally
```

```
// Thực tế, nó không đơn giản như vậy.
```

```
// resolve() còn thay đổi [[PromiseState]] (trạng thái của Promise) thành fulfilled (thành công)
```

```
// reject() còn thay đổi [[PromiseState]] thành rejected (thất bại)
```

Xử lý bất đồng bộ bằng promise

API là gì : thông thường ở bên BE sẽ tạo ra API cho bên phía FE sử dụng hiểu đơn giản API là những câu lệnh, function, giao thức, object, arr...

=> giúp FE và BE có thể tương tác và trao đổi với dữ liệu đc với nhau

- giao thức: GET, POST, PUT, DELETE, PATCH.

- + GET: Lấy dữ liệu từ máy chủ. (Không có tác động phụ đến dữ liệu trên máy chủ. Thường dùng để lấy thông tin.)

- + POST: Tạo mới tài nguyên.

- + PUT: Cập nhật một tài nguyên hiện có trên máy chủ (Thay thế hoàn toàn tài nguyên hiện tại.)

- + DELETE: Xóa một tài nguyên trên máy chủ.

- + PATCH: Cập nhật một phần của tài nguyên hiện có trên máy chủ.

Xử lý bất đồng bộ bằng promise

VÍ DỤ ỨNG DỤNG PROMISE CỤ THỂ

Call API get post : <https://jsonplaceholder.typicode.com/posts>

Data trả về

```
▼ [,...]  
  ▶ 0: {userId: 1, id: 1, title: "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",...}  
  ▶ 1: {userId: 1, id: 2, title: "qui est esse",...}  
  ▶ 2: {userId: 1, id: 3, title: "ea molestias quasi exercitationem repellat qui ipsa sit aut",...}  
  ▶ 3: {userId: 1, id: 4, title: "eum et est occaecati",...}  
  ▶ 4: {userId: 1, id: 5, title: "nesciunt quas odio",...}  
  ▶ 5: {userId: 1, id: 6, title: "dolorem eum magni eos aperiam quia",...}  
  ▶ 6: {userId: 1, id: 7, title: "magnam facilis autem",...}  
  ▶ 7: {userId: 1, id: 8, title: "dolorem dolore est ipsam",...}
```

```

4
5 const getPosts = new Promise((resolve, reject) => {
6   const req = new XMLHttpRequest(); // Tạo một đối tượng XMLHttpRequest
7   req.open("GET", "https://jsonplaceholder.typicode.com/posts"); // Thiết lập yêu cầu
8   req.send(); // gửi 1 yêu cầu gọi api
9   req.onload = () => {
10     if (req.status === 200) {
11       resolve(req.responseText);
12     } else {
13       reject();
14     }
15   };
16 });
17
18 getPosts
19   .then((data) => {
20     console.log(data, "data");
21   })
22   .catch((error) => {
23     console.error(error);
24   });
25

```

Chúng ta sẽ chủ yếu thao tác với một promise có sẵn, được tạo ra bởi người khác từ các thư viện hoặc các API có sẵn của JS, tuy nhiên ta cũng có thể tự tạo ra một Promise để làm những logic cần thiết

Xử lý bất đồng bộ bằng promise

Chain promise :

để giải quyết vấn đề callbackhell. thì promise chain xin ra để giải quyết vấn đề đó .

```
doSomething(function (result) {  
  doSomethingElse(result, function (newResult) {  
    doThirdThing(newResult, function (finalResult) {  
      console.log(`Got the final result: ${finalResult}`);  
    }, failureCallback);  
  }, failureCallback);  
}, failureCallback);
```



```
doSomething()  
  .then(function (result) {  
    return doSomethingElse(result);  
  })  
  .then(function (newResult) {  
    return doThirdThing(newResult);  
  })  
  .then(function (finalResult) {  
    console.log(`Got the final result: ${finalResult}`);  
  })  
  .catch(failureCallback);
```

arrow function

```
doSomething()  
  .then((result) => doSomethingElse(result))  
  .then((newResult) => doThirdThing(newResult))  
  .then((finalResult) => {  
    console.log(`Got the final result: ${finalResult}`);  
  })  
  .catch(failureCallback);
```

Xử lý bất đồng bộ bằng promise

GỌI API DÙNG FETCH

77. GUY ARI DANG TEECH

```
fetch("https://jsonplaceholder.typicode.com/todos")
  .then((res) => {
    console.log(res);
    return res.json()
  })
  .then((data) => {
    console.log(data);
  })
```

```
(200) [{"-","-","-","-","-","-","-","-","-","-","-","-","-","-","-","-","-","-","-","-"},
{"-","-","-","-","-","-","-","-","-","-","-","-","-","-","-","-","-","-","-","-"},
{"-","-","-","-","-","-","-","-","-","-","-","-","-","-","-","-","-","-","-","-"},
{"-","-","-","-","-","-","-","-","-","-","-","-","-","-","-","-","-","-","-","-"}]
▼ [0 ... 99]
  0: {userId: 1, id: 1, title: 'delectus aut autem', completed: false}
  1: {userId: 1, id: 2, title: 'quis ut nam facilis et officia qui', completed: false}
  2: {userId: 1, id: 3, title: 'fugiat veniam minus', completed: false}
  3: {userId: 1, id: 4, title: 'et porro tempora', completed: true}
  4: {userId: 1, id: 5, title: 'laboriosam mollitia et enim quasi adipisci quia provident illum', completed: true}
  5: {userId: 1, id: 6, title: 'qui ullam ratione quibusdam voluptatem quia omnis', completed: false}
  6: {userId: 1, id: 7, title: 'illo expedita consequatur quia in', completed: false}
  7: {userId: 1, id: 8, title: 'quo adipisci enim quam ut ab', completed: true}
  8: {userId: 1, id: 9, title: 'molestiae perspiciat ipsa', completed: false}
  9: {userId: 1, id: 10, title: 'illo est ratione doloremque quia maiores aut', completed: true}
  10: {userId: 1, id: 11, title: 'vero rerum temporibus dolor', completed: true}
  11: {userId: 1, id: 12, title: 'ipsa repellendus fugit nisi', completed: true}
  12: {userId: 1, id: 13, title: 'et doloremque nulla', completed: false}
  13: {userId: 1, id: 14, title: 'repellendus sunt dolores architecto voluptatum', completed: true}
  14: {userId: 1, id: 15, title: 'ab voluptatum amet voluptas', completed: true}
  15: {userId: 1, id: 16, title: 'accusamus eos facilis sint et aut voluptatem', completed: true}
  16: {userId: 1, id: 17, title: 'quo laboriosam deleniti aut qui', completed: true}
  17: {userId: 1, id: 18, title: 'dolorem est consequatur ea mollitia in culpa', completed: false}
```

Xử lý bất đồng bộ bằng ASYNC/AWAIT

- ❖ Async/await là cú pháp mới để xử lý các hàm bất đồng bộ có dạng promise , thay thế cho cú pháp .then, catch trước đó
- ❖ Cú pháp này đưa code bất đồng bộ của ta về gần giống như code đồng bộ thông thường ta viết , khiến mọi thứ dễ dàng kiểm soát cho hầu hết trường hợp

```
async function name(param0) {  
  statements  
}  
  
async function name(param0, param1) {  
  statements  
}  
  
async function name(param0, param1, /* ... ,*/ paramN) {  
  statements  
}
```